

Database Lab Session 2: Empirical Evaluation - Relational DBMS vs. NoSQL Document DBMS

Sergio Peignier and Nicolas Parisot

1 Objectives and Context

The core objective of this practical evaluation is to perform a rigorous performance and financial comparison between a Relational Database Management System (RDBMS) and a Document-based NoSQL database engine. Specifically, we aim to quantify the cost of data normalization by measuring the mechanical behavior, processing overhead, and execution latency of relational multi-table JOIN operations connecting hospital health records with large public safety surveillance datasets, compared to highly denormalized, nested document retrievals.

To simulate production clinical workloads, you will construct a Python-based benchmarking pipeline using local database engine layers:

- **Relational (SQL):** A localized SQLite database engine designed to mimic a structured, normalized relational transaction environment.
- **Non-Relational (NoSQL):** A JSON-based local document dictionary store optimized for nested lookups.

2 Dataset Architecture

The unified health-safety dataset is formatted using two distinct architectural models to emphasize the structural layout divergence:

Format A: The Relational Architecture (`hospital_fda_registry.sql`) The relational schema is normalized into two distinct tables linked by a common domain attribute (`Medication` ↔ `suspect_drug`) to prevent storage duplication across massive tracking registries:

- **Table 1: Patients**
(`Patient_ID` [PK], `sex`, `age`, `date_of_birth`, `address`, `postal_code`, `Height_cm`, `Weight_kg`, `BMI`, `Smoking_Status`, `alcohol_consumption`, `physical_activity`, `blood_pressure`, `Cholesterol_mg_dL`, `Glucose_mg_dL`, `Medical_Diagnosis`, `Genomic_Variant`, `Medication` [FK])
- **Table 2: FDA_Adverse_Events**
(`report_id` [PK], `receive_date`, `year`, `month`, `quarter`, `serious`, `serious_flags`, `is_fatal`, `is_hospitalized`, `is_life_threat`, `is_disabling`, `reactions`, `primary_reaction`, `reaction_outcomes`, `patient_recovered`, `num_reactions`, `suspect_drug` [FK], `brand_name`, `drug_route`, `drug_indication`, `manufacturer`, `pharm_class`, `num_drugs`, `drug_count_category`, `patient_age_years`, `age_group`, `patient_sex`, `patient_weight_kg`, `country`, `report_age_days`)

Format B: The NoSQL Document Architecture (`hospital_fda_registry.json`) The document schema is completely denormalized to optimize read throughput. The database eliminates relational boundaries entirely. Each patient record is stored as a self-contained JSON document. The matching FDA safety data (including all metadata fields such as `report_id`, `reactions`, seriousness flags, and drug manufacturing details) is fully embedded as an array of child objects directly inside the parent patient document under an `"fda_alerts"` key.

2.1 Exercise 1: Implementing the Benchmarking Pipeline

Construct a Python automated benchmarking script named `db.benchmark.py`. The script will utilize `pandas` for data ingestion, map the tables' records into an relational SQLite database, and construct a denormalized nested NoSQL document store out of the data layers.

Follow these algorithmic and structural requirements to complete the task:

1. Environmental Setup and Ingestion:

- Import `sqlite3`, `pandas` (as `pd`), and `time`.
- Load `patients.csv` and `fda_adverse_events.csv` into two independent Pandas DataFrames (`df_patients` and `df_fda`).

2. Data Cleaning:

- How would you audit the columns allowing to JOIN both tables (i.e. `Medication` and `suspected_drug`) to determine why a direct `pd.merge()` fails?
- Implement a custom transformation to solve the problems and verify that your transformation successfully established overlapping elements between both columns.

3. Relational Database Mapping (SQL Tier):

- Establish a SQLite connection via `sqlite3.connect('yourDatabaseName.sql')`. If you do not want the database to be persistent, you can create an in-memory database via `sqlite3.connect(':memory:')`
- Use the native Pandas method `.to_sql()` to automatically write both DataFrames into separate SQLite tables named `Patients` and `FDA_Adverse_Events`. Ensure index columns are omitted.

4. Data Transformation and Embedding (NoSQL Document Tier):

- Replicate a document-oriented NoSQL collection by transforming the data frames into a nested structure in-memory.
- **Grouping Vector:** Use the Pandas `.groupby()` method on `df_fda`, targeting the `suspect_drug` field. Apply a lambda function to convert grouped tables into lists of records via `.to_dict(orient='records')`.
- **Nesting Phase:** Convert `df_patients` to a dictionary keyed by `Patient_ID`. Iterate through the patient records and programmatically embed the corresponding array of FDA safety alerts inside a new child field named `"fda_alerts"`. Store the output in a global variable named `nosql_document_store`. You can save the document using the `json` Python library.

5. The Performance Test Harness:

- Define a list of **20** random target patient for lookup metric tracking.
- **Relational Loop:** Execute a loop for each patient. Inside the loop, run a parameterized `LEFT JOIN` query linking `Patients` and `FDA_Adverse_Events` on the medication criteria, filtered by the patient target ID. Use `time.time()` to record total execution delta.
- **Document Loop:** Execute a loop over the target patients that performs a direct key lookup against `nosql_document_store`, extracting the targeted patient document and accessing its nested fields instantly without a relational calculation. Record the execution time delta.

6. Telemetry Logging:

- Print the total execution duration for both the SQL approach and the NoSQL document approach.
- **Tracking Memory Footprints (RAM Allocation):**
 - Utilize the built-in Python library `tracemalloc` to capture structural heap memory shifts.
 - Call `tracemalloc.start()` immediately before entering a test loop, and use `tracemalloc.get_traced_memory()` immediately upon loop termination to extract the peak memory usage in megabytes (MB). Ensure `tracemalloc.stop()` is called between loops to clear the counter buffer.
- Programmatically calculate and output the exact performance scaling ratio (e.g., *"NoSQL Document Read is X.Xx faster than SQL JOIN operations"*).

2.2 Costs Forecast and DBMS Choice

Beyond measuring raw programmatic latency, an engineer must translate infrastructure behaviors into a predictable Total Cost of Ownership forecast. Your task is to compose a formal, written engineering defense responding to the following production deployment scenario:

Production Scenario: The hospital network plans to expand its core tracking infrastructure to accommodate 200,000 active patient profiles over the next 2 years, with automated background pipelines ingesting hundreds of thousands of public FDA safety alerts. Analysis of operational traffic patterns indicates that 95% of runtime system transactions consist of single-point clinical lookups where a physician pulls up a specific patient profile to instantly review their history alongside any embedded FDA warning/reaction flags for their current prescription. Conversely, the remaining 5% of traffic involves epidemiology researchers running complex cross-table analytical queries to correlate global metrics (e.g., mapping whether patient lifestyle factors like `Smoking_Status` and `BMI` show statistical correlations with the severity flags or `reaction_outcomes` found inside the FDA registry).

Based on your local laboratory metrics and the provider pricing sheet provided (document 1), which database architecture do you select as your primary production storage solution, and how do you justify the resulting technical and financial trade-offs?

To do this, using the *Cloud Infrastructure Cost Specification Sheet* (document 1), calculate monthly operational expenses by comparing two entirely divergent billing paradigms:

- **Relational Tier:** Costs scale by raw hardware allocation (vCPUs and RAM) necessary to absorb computational resource spikes during heavy multi-table query-time `JOIN` execution loops, combined with base block storage volumes and provisioned IOPS¹.

¹Input Output Operations Per Second

- **NoSQL Tier:** Costs decouple from fixed machine sizes and instead scale directly by uncompressed data footprint size and provisioned throughput capacity units dictated by object size parameters.

Utilizing the operational metrics recorded in your active laboratory findings, complete the following financial engineering tasks to establish your baseline monthly cost projections for Table 1.

Table 1: Engineering Performance Metrics Table

Performance Metric	Relational Model (SQL)	Document Model (JSON)
Avg. Response Time per Request	<i>Empirical calculation (ms)</i>	<i>Empirical calculation (ms)</i>
Max Queries Per Minute (QPM)	<i>Estimated capacity</i>	<i>Estimated capacity</i>
Memory Behavior Scale	CPU and RAM empirical use	CPU and RAM empirical use
Estimated Cost (Cloud)	<i>Provided tier</i>	<i>Provided tier</i>

1. **Calculate Relational Core Demand:** Analyze your local CPU metrics and choose among the Relational Database Compute Tier Options. Compute the monthly compute cost across a standard billing cycle (24 hours $\times$ 30.5 days), adding the flat relational storage volume consumption and the minimum required IOPS performance floor.
2. **Compute Throughput Volume Capacity:** Compute the read volume requirement (requests) per second needed to sustain the target production workload of 10,000 requests per minute.
3. **Determine the Payload Multiplier:** Consider your denormalized JSON data layout, and determine your average document payload size multiplier based on how heavily nested the embedded FDA safety arrays are (where 1 RCU² covers a block up to 4 KB in size).
4. **Project Monthly NoSQL Utility Consumption:** Multiply the required operational RCU computed in the previous question by the hourly base utility cost rate per provisioned RCU. Scale this throughput figure across a full month (24 hours $\times$ 30.5 days), and append the flat monthly document storage footprint rate (\$0.25 per GB for the 120 GB collection size) to achieve the final NoSQL monthly cost projection.

²Read Capacity Unit covering up to **1 strongly consistent read per second** for a document up to **4 KB** in size.

SYSTEM DOSSIER: DOCUMENT 1

Title: Cloud Infrastructure Cost & Provisioning Specification Sheet

Source: Enterprise Cloud Architecture & FinOps Division

Status: Standardized Pricing Matrix (FY26/27)

3

This reference sheet outlines the production hosting costs for both infrastructure pathways on our cloud provider platform. Students must use their local data metrics, dataset sizing calculations, and transaction latency footprints to compute the projected Monthly Total Cost of Ownership (TCO) required for your laboratory findings.

4 Production Architecture Baseline Parameters

- **Target System Scale:** 200,000 active patient profiles + ingested FDA historical registries.
- **Calculated Total Document Dataset Footprint: 120 GB** (Uncompressed JSON collection size).
- **Calculated Total Relational Dataset Footprint: 45 GB** (Optimized, indexed SQLite/-PostgreSQL equivalent storage).
- **Operational Traffic Assumptions:** The database must comfortably handle a base workload of **10,000 requests per minute** (95% point lookups, 5% complex analytical aggregations).

5 Tier 1: Relational Database Provisioning (Managed Instance Model)

Relational database scaling is fundamentally tied to fixed hardware provisioning (Compute + Allocated Block Storage Volumes). High CPU utilization spikes during heavy nested JOIN execution loops require a database instance with dedicated hardware cores to prevent connection queue line blocks.

Table 2: Relational Database Compute Tier Options

Instance Tier	Target Metric Suitability	vCPUs	RAM	Cost US\$
db.m6g.large	Low CPU stress, minimal lookups	2	8 GB	\$0.120 / hr
db.m6g.xlarge	Moderate CPU load, moderate joins	4	16 GB	\$0.240 / hr
db.m6g.2xlarge	High CPU loops / Heavy relational computation	8	32 GB	\$0.480 / hr

5.1 Relational Storage Volumes (EBS SSD)

- **Base Storage Cost:** \$0.115 per GB per Month.

- **Provisioned IOPS Base (Required for relational index trees):** Fixed \$0.05 per Provisioned IOPS per Month (Minimum recommended: 3,000 IOPS for performance testing stability).

In order to determine the most suitable tier option, you should analyze your local CPU metrics. If your local SQL execution loop registered high processor overhead, you must select the high-compute `db.m6g.2xlarge` instance tier for your calculations. Combine the monthly compute cost (24 hours $\times$ 30.5 days) with your calculated relational storage footprint and IOPS floor to find the total monthly expenditure.

6 Tier 2: NoSQL Document Storage Provisioning (Utility Sizing Model)

Document-oriented NoSQL pricing completely bypasses fixed instance hardware specs. Instead, it bills purely based on storage footprint size and raw transactional throughput capacity units.

6.1 1. Storage Base Footprint Cost

- **Standard Document Storage Rate:** \$0.25 per GB per Month.

6.2 2. Provisioned Transactional Throughput Capacity

NoSQL systems isolate throughput costs into independent Read Capacity Units (RCUs) and Write Capacity Units (WCUs):

- **Read Capacity Unit (RCU) Definition:** 1 RCU covers up to **1 strongly consistent read per second** for a document up to **4 KB** in size.
- For example if a denormalized patient JSON document scales up to **24 KB** due to heavily embedded FDA nested arrays, fetching that single document requires $\frac{24 \text{ KB}}{4 \text{ KB}} = 6$ RCUs per read request.
- **RCU Cost Rate:** \$0.00013 per hour per provisioned RCU block.